

CMU 10-703: Deep Reinforcement Learning

Lecture 6: Value-Based Methods II

Max Simchowitz

Carnegie Mellon University

1.

DQN Foundations

CMU 10-703: Deep Reinforcement Learning

Fitted Q-Iteration Recap

Bellman target from the previous fit

$$y_i(s, a) = r(s, a) + \gamma \max_{a'} Q_{\theta_i}(s', a')$$

Fitted Q-iteration converts a Bellman backup into a supervised target.

Fitted Q-Iteration Recap

Bellman target from the previous fit

$$y_i(s, a) = r(s, a) + \gamma \max_{a'} Q_{\theta_i}(s', a')$$

Regression update

$$\theta_{i+1} \leftarrow \arg \min_{\theta} \mathbb{E}_{(s,a,s') \sim \mu} \left[(Q_{\theta}(s, a) - y_i(s, a))^2 \right]$$

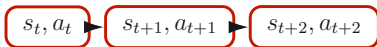
The sampling distribution μ may contain transitions from policies other than the current greedy policy.

Off-policy data reuse is possible, but the fitted values are reliable only where the data provide coverage.

Deep Function Approximation

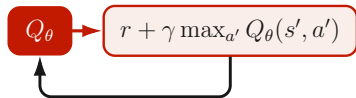
Correlated samples

Consecutive transitions from an ϵ -greedy rollout are similar. Training only on the latest trajectory can overweight recent experience and forget older behavior.



Moving targets

Updating Q_θ also changes the Bellman target. The network chases a regression target produced by its own changing parameters.



DQN adds replay and delayed targets to make neural Bellman regression less unstable.

Epsilon-Greedy Exploration

At iteration i , the current Q-function defines the behavior policy:

$$a = \begin{cases} \text{Uniform}(\mathcal{A}) & \text{with probability } \epsilon, \\ \arg \max_{a'} Q_{\theta_i}(s, a') & \text{with probability } 1 - \epsilon. \end{cases}$$

Greedy component

Biases data collection toward states reached by policies that already obtain reasonable returns.

Random component

Explores actions around those states and can reveal unseen state-action pairs. A typical fixed value is $\epsilon = 0.1$.

Purely random behavior can diffuse into poor states; occasional random actions explore around the current policy's useful behavior.

DQN Components

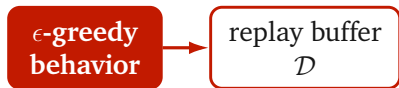
ϵ -greedy
behavior

Explore

Collect useful and
varied actions.

The behavior policy generates the stream of experience used for learning.

DQN Components



Explore

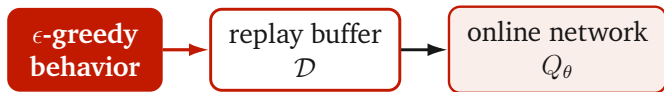
Collect useful and varied actions.

Store

Retain recent experience.

A replay buffer retains experience across time and policy versions.

DQN Components



Explore

Collect useful and varied actions.

Store

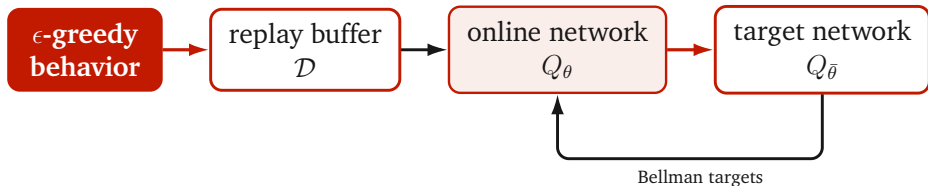
Retain recent experience.

Fit

Minimize Bellman error.

The online network learns from randomized minibatches of stored transitions.

DQN Components



Explore

Collect useful and varied actions.

Store

Retain recent experience.

Fit

Minimize Bellman error.

Stabilize

Slow target changes.

A delayed target network supplies the bootstrap values used in regression.

2.

Replay, Targets, and Data Reuse

CMU 10-703: Deep Reinforcement Learning

Replay Buffer Lifecycle

collect
 (s, a, r, s')

Environment interaction produces one transition at a time.

Replay Buffer Lifecycle



When the circular buffer is full, a new transition replaces the oldest stored transition.

A finite circular buffer keeps a moving history of recent experience.

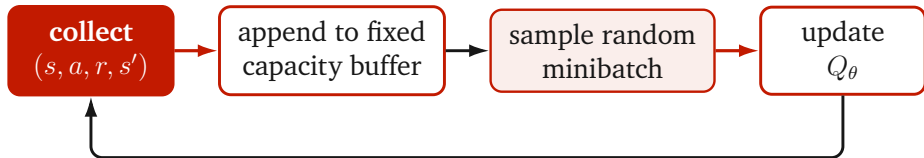
Replay Buffer Lifecycle



When the circular buffer is full, a new transition replaces the oldest stored transition.

Random minibatches mix transitions collected at different times.

Replay Buffer Lifecycle



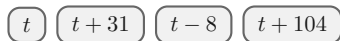
When the circular buffer is full, a new transition replaces the oldest stored transition.

Replay lets one transition support several gradient updates before it leaves the buffer.

Two Roles of Experience Replay

Decorrelation

Random sampling breaks the adjacency of a single trajectory. A minibatch can combine transitions from distant times, episodes, and policy versions.



Revisiting experience

Stored transitions remain available after the behavior policy changes. Repeated sampling improves data use and reduces rapid forgetting of older but relevant behavior.



Replay changes both the dependence structure of minibatches and the number of times each transition can influence learning.

DQN Targets and Loss

Target-network Bellman target

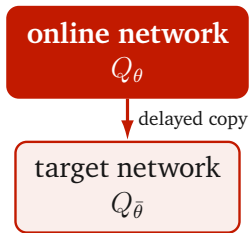
$$y = \begin{cases} r & \text{if } s' \text{ is terminal,} \\ r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') & \text{otherwise.} \end{cases}$$

Replay-distribution regression loss

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(Q_{\theta}(s, a) - y)^2 \right]$$

The online network predicts the sampled action value; the delayed network supplies only the nonterminal bootstrap term.

Target Network Delay



Fast time scale

Gradient descent updates θ on every optimization step.

Slow time scale

The target parameters $\bar{\theta}$ remain fixed for several steps or move by a small amount.

Optimization effect

A minibatch sees a consistent target while the online prediction moves toward it.

Delaying the target reduces feedback between the parameters being optimized and the values used to supervise them.

Hard and Soft Target Updates

Hard update

Hold the target fixed, then copy the online parameters every k steps.

$$\bar{\theta} \leftarrow \theta \quad \text{every } k \text{ steps}$$

Large k produces longer intervals with an unchanged target.

Soft update

Move the target toward the online parameters after every gradient step.

$$\bar{\theta}_{i+1} \leftarrow \alpha \theta_{i+1} + (1 - \alpha) \bar{\theta}_i$$

A typical small update rate is $\alpha = 0.001$.

Large hard-update intervals and small soft-update rates both separate the target network from rapid online updates.

Updates-to-Data Ratio

Updates-to-data ratio. The number of gradient updates performed relative to the amount of new experience collected.

$$\text{UTD} = \frac{\text{number of gradient updates}}{\text{amount of new data collected in one rollout}}$$

High UTD

Reuses each data point many times, but can overfit to a limited replay distribution.

Low UTD

Collects substantial new experience between updates, but may leave useful stored data underused.

UTD controls the balance between environment interaction and optimization on stored data.

UTD Scale and Buffer Turnover

Illustrative turnover

Suppose the replay capacity is 200,000 transitions and the agent collects 400,000 new timesteps before updating. The buffer is replaced before much of its contents can be reused.

Interpretation

This regime has a very small UTD ratio.

Practical scale

Values between 10^{-3} and 100 are common, with the useful range depending on the environment, task, buffer, and function approximator.

No universal optimum

More updates can improve data use or amplify fitting errors.

The same nominal buffer can behave very differently depending on how quickly new data replace old data and how often each sample is revisited.

DQN Algorithm I: Initialization and Interaction

Algorithm 1: DQN with Experience Replay

- 1: Initialize replay memory $\mathcal{D}$ to capacity N .
- 2: Initialize the action-value function Q with random weights θ .
- 3: Initialize the target action-value function $\hat{Q}$ with weights $\bar{\theta} = \theta$.
- 4: **for** episode = $1, \dots, M$ **do**
- 5: **for** $t = 1, \dots, T$ **do**
- 6: With probability ϵ , select a random action a_t .
- 7: Otherwise, select $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$.
- 8: Execute a_t and observe reward r_t and state s_{t+1} .
- 9: Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$.

DQN continually adds epsilon-greedy experience to a finite replay memory.

DQN Algorithm II: Targets and Updates

Algorithm 1 continued

- 10: Sample a random minibatch (s_j, a_j, r_j, s_{j+1}) of size K from $\mathcal{D}$.
11: Set the target for each sampled transition:

$$y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal,} \\ r_j + \gamma \max_{a'} \widehat{Q}(\phi(s_{j+1}), a'; \bar{\theta}) & \text{otherwise.} \end{cases}$$

- 12: Take a gradient step on the squared error:

$$(y_j - Q(\phi(s_j), a_j; \theta))^2$$

- 13: Every C steps, update the target network: $\bar{\theta} \leftarrow \theta$.
14: **end for**
15: **end for**

3.

Overestimation and Instability

CMU 10-703: Deep Reinforcement Learning

The Deadly Triad

Function approximation

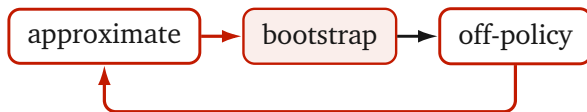
A neural network couples value estimates across many state-action pairs.

Bootstrapping

The target contains a value estimate produced by another Q-function evaluation.

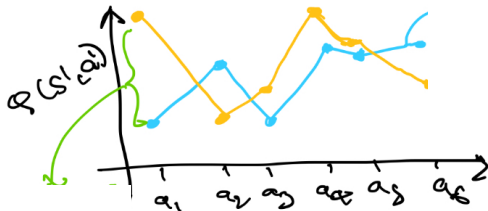
Off-policy learning

Replay data may come from earlier or different behavior policies.



Each ingredient is useful, but their combination can produce unstable or divergent Q-learning updates.

Overestimation in the Bellman Target



Blue curve

True action values.

Yellow curve

Noisy target estimates.

Green gap

Maximization selects a high estimation error and may also select the wrong action.

Taking a maximum over noisy values tends to select an action whose estimate is too large.

Sources of Overestimation

Incomplete coverage

The Q-network may have seen little data for some state-action pairs, so its errors differ substantially across actions.

Maximum over errors

The target uses the action with the largest estimate, which can be large because the action is good or because its error is positive.

Biased supervision

An optimistic maximum becomes the regression target for the current state-action pair.

Network coupling

Fitting the biased target changes estimates at other inputs, so the error can spread beyond the sampled transition.

Maximization turns uneven approximation error into systematically optimistic Bellman targets.

Error Compounding

A mistaken value estimate can generate the next target, which trains the next estimate and then generates another target:

$$\begin{aligned}\theta_0 \text{ (erroneous)} &\longrightarrow y_0 \text{ (erroneous)} \longrightarrow \theta_1 \text{ (erroneous)} \\ &\longrightarrow y_1 \text{ (erroneous)} \longrightarrow \dots \longrightarrow \theta_k^{\text{target}} \text{ (erroneous)}\end{aligned}$$

A high UTD ratio does not create maximization bias by itself. Repeated updates on the same limited data can amplify an initial overestimate through overfitting and repeated bootstrapping.

Biased targets can propagate through successive network and target updates; aggressive data reuse can accelerate the propagation.

4.

Double Q-Learning

CMU 10-703: Deep Reinforcement Learning

Action Selection and Evaluation

Double Q-learning. Maintain differently initialized estimators and use one to select an action while the other evaluates it.

$$a^* = \arg \max_a Q^A(s', a)$$

The first estimator chooses the next action.

Source: Hasselt, "Double Q-learning," NeurIPS 23 (2010).

Action Selection and Evaluation

Double Q-learning. Maintain differently initialized estimators and use one to select an action while the other evaluates it.

$$a^* = \arg \max_a Q^A(s', a)$$

$$y = r + \gamma Q^B(s', a^*)$$

The estimation error that selected a^* does not directly determine the value assigned to that action.

The second estimator evaluates the selected action, reducing the coupling that produces maximization bias.

Source: Hasselt, "Double Q-learning," NeurIPS 23 (2010).

Tabular Double Q-Learning I

Algorithm: Double Q-learning

- 1: Initialize Q^A , Q^B , and state s .
- 2: **repeat**
- 3: Choose a based on $Q^A(s, \cdot)$ and $Q^B(s, \cdot)$; observe r, s' .
- 4: Choose, for example at random, either UPDATE(A) or UPDATE(B).
- 5: **if** UPDATE(A) **then**
- 6: Define the action selected by estimator A:

$$a^* = \arg \max_{a'} Q^A(s', a')$$

- 7: Update estimator A using estimator B:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(s, a) (r + \gamma Q^B(s', a^*) - Q^A(s, a))$$

Tabular Double Q-Learning II

Algorithm continued

- 8: **else if** UPDATE(B) **then**
9: Define the action selected by estimator B:

$$b^* = \arg \max_{a'} Q^B(s', a')$$

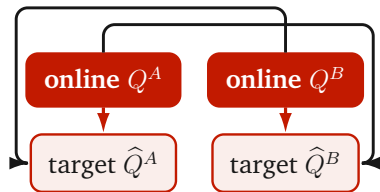
- 10: Update estimator B using estimator A:

$$Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(s, a) (r + \gamma Q^A(s', b^*) - Q^B(s, a))$$

- 11: **end if**
12: $s \leftarrow s'$
13: **until end**

Source: Faithful typesetting of the lecture-notes figure; Hasselt (2010).

Deep Double Q-Learning Structure



Behavior action

The source algorithm acts greedily with the sum of the two online estimates, apart from epsilon exploration.

$$a_t = \arg \max_a (Q^A(\phi(s_t), a; \theta^A) + Q^B(\phi(s_t), a; \theta^B))$$

Each online estimator selects actions for targets evaluated by the other estimator's delayed target network.

Deep Double Q Algorithm I: Interaction

Algorithm 2: Deep Double Q-learning with Random Update

- 1: Initialize replay memory $\mathcal{D}$ to capacity N .
- 2: Initialize online networks Q^A, Q^B with random weights θ^A, θ^B .
- 3: Initialize target networks $\hat{Q}^A, \hat{Q}^B$ with $\theta^{A-} = \theta^A$ and $\theta^{B-} = \theta^B$.
- 4: **for** episode = $1, \dots, M$ **do**
- 5: **for** $t = 1, \dots, T$ **do**
- 6: With probability ϵ , select a random action a_t .
- 7: Otherwise, select

$$a_t = \arg \max_a \left(Q^A(\phi(s_t), a; \theta^A) + Q^B(\phi(s_t), a; \theta^B) \right)$$

- 8: Execute a_t and observe r_t, s_{t+1} .
- 9: Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$.

Deep Double Q Algorithm II: Update A

Algorithm 2 continued

- 10: Sample a random minibatch (s_j, a_j, r_j, s_{j+1}) from $\mathcal{D}$.
- 11: **if** updating A, with probability 0.5, **then**
- 12: Select the next action with online network A:

$$a' = \arg \max_{a'} Q^A(\phi(s_{j+1}), a'; \theta^A)$$

- 13: Evaluate it with target network B:

$$y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal,} \\ r_j + \gamma \widehat{Q}^B(\phi(s_{j+1}), a'; \theta^{B-}) & \text{otherwise.} \end{cases}$$

- 14: Take a gradient step on $(y_j - Q^A(\phi(s_j), a_j; \theta^A))^2$ to update θ^A .

Deep Double Q Algorithm III: Update B

Algorithm 2 continued

15: **else**, update network B.

16: Select the next action with online network B:

$$b' = \arg \max_{a'} Q^B(\phi(s_{j+1}), a'; \theta^B)$$

17: Evaluate it with target network A:

$$y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal,} \\ r_j + \gamma \hat{Q}^A(\phi(s_{j+1}), b'; \theta^{A-}) & \text{otherwise.} \end{cases}$$

18: Take a gradient step on $(y_j - Q^B(\phi(s_j), a_j; \theta^B))^2$ to update θ^B .

19: **end if**

20: Every C steps, set $\theta^{A-} \leftarrow \theta^A$ and $\theta^{B-} \leftarrow \theta^B$.

21: **end for**

22: **end for**

5.

Multi-Step Returns

CMU 10-703: Deep Reinforcement Learning

N-Step Bellman Targets

Replace a one-step bootstrap with several observed rewards before applying the target-network maximum:

$$y(s_t, a_t) = r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \dots \\ + \gamma^N \max_{a_{N+1}} Q^{\text{target}}(s_{N+1}, a_{N+1}).$$

The coefficient on the maximized target becomes γ^N , reducing the contribution of estimation error in the bootstrap term.

A longer return delays bootstrapping and discounts the noisy maximum more heavily.

N-Step Return Tradeoff

Reduced max dependence

More observed rewards enter before the target-network maximum. The γ^N factor attenuates error in the final bootstrap value.

Less compounding

Fewer successive one-step bootstrap targets intervene between the current transition and later rewards.

Longer returns reduce reliance on an estimated maximum but introduce greater dependence on the replayed behavior sequence.

Off-policy mismatch

Intermediate states and actions come from replayed behavior and can differ from actions chosen by the optimal greedy policy.

Objective tradeoff

The target moves away from the one-step optimality backup that directly applies $\max_{a'} Q^{\text{target}}(s', a')$.

Stabilization Mechanisms

Mechanism	Direct purpose	Remaining tradeoff
Experience replay	Decorrelate samples and reuse past transitions	The replay distribution may have poor coverage
Target network	Slow changes in the Bellman regression target	Delayed values can still be biased
Double Q-learning	Separate action selection from evaluation	Requires multiple estimators and update schedules
N-step returns	Reduce weight on the maximized bootstrap value	Depend more on off-policy intermediate actions

Each mechanism controls a different source of instability; none removes approximation, coverage, and data-reuse concerns at once.

Lecture Summary

DQN mechanics

Epsilon-greedy interaction, replay, and delayed targets turn fitted Q-iteration into an incremental deep-learning algorithm.

Overestimation

Maximizing noisy action values can create optimistic targets that compound through repeated bootstrapping.

Bias control

Double Q-learning separates selection from evaluation; n-step returns reduce reliance on the final maximized value.

stable value learning = data control + target control + bias control

Reliable deep Q-learning requires control over what data are reused, how quickly targets move, and how maximization error enters those targets.